8 puzzle problem

Program:

```
from collections import deque

def solve(start, goal):
    queue = deque([(start, [])])
    visited = {tuple(start)}

    while queue:
        state, path = queue.popleft()

        if state == goal:
            for step in path + [state]:
                print(step[:3])
                print(step[3:6])
                print(step[6:])
                print()
            return

        zero = state.index(0)
        row, col = divmod(zero, 3)

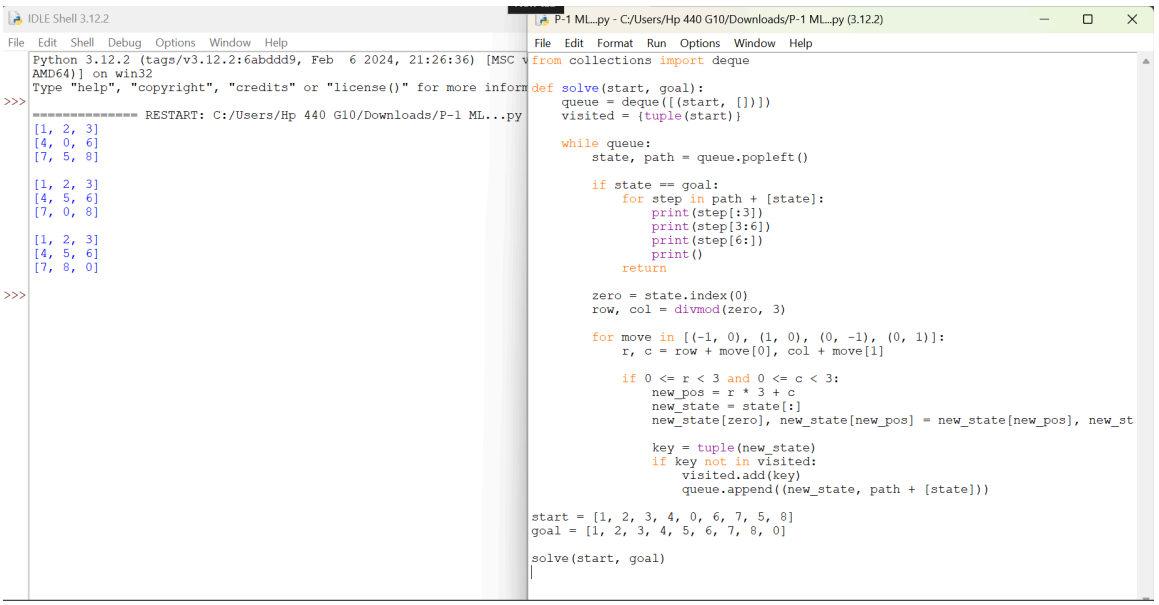
        for move in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            r, c = row + move[0], col + move[1]

            if 0 <= r < 3 and 0 <= c < 3:
                new_pos = r * 3 + c
                new_state = state[:]
                new_state[zero], new_state[new_pos] = new_state[new_pos], new_state[zero]

                key = tuple(new_state)
                if key not in visited:
                    visited.add(key)
                    queue.append((new_state, path + [state]))

start = [1, 2, 3, 4, 0, 6, 7, 5, 8]
goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]

solve(start, goal)
```


IDLE Shell 3.12.2
File Edit Shell Debug Options Window Help
Python 3.12.2 (tags/v3.12.2:6abddd9, Feb 6 2024, 21:26:36) [MSC v
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more inform
>>>
========== RESTART: C:/Users/Hp 440 G10/Downloads/P-1 ML...py
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
>>>
P-1 ML...py - C:/Users/Hp 440 G10/Downloads/P-1 ML...py (3.12.2)
File Edit Format Run Options Window Help
from collections import deque
def solve(start, goal):
queue = deque([(start, [])])
visited = {tuple(start)}
while queue:
state, path = queue.popleft()
if state == goal:
for step in path + [state]:
print(step[:3])
print(step[3:6])
print(step[6:])
print()
return
zero = state.index(0)
row, col = divmod(zero, 3)
for move in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
r, c = row + move[0], col + move[1]
if 0 <= r < 3 and 0 <= c < 3:
new_pos = r * 3 + c
new_state = state[:]
new_state[zero], new_state[new_pos] = new_state[new_pos], new_st
key = tuple(new_state)
if key not in visited:
visited.add(key)
queue.append((new_state, path + [state]))
start = [1, 2, 3, 4, 0, 6, 7, 5, 8]
goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]
solve(start, goal)